

Lab 5 - Tasks for Students

Task: Implement Language Selector using React Context

Objective

Create a global language selection system using React Context that allows users to select their preferred language (English or Spanish) in the Settings page, and have this selection persist across the entire application.

Prerequisites

- Basic understanding of React Hooks (`useState`, `useContext`)
 - Understanding of React Context API
 - Familiarity with React Native components
-

Task Breakdown

Task 1: Create LanguageContext

Location: `contexts/LanguageContext.js`

Requirements:

1. Create a new file `LanguageContext.js` in the `contexts` folder
2. Import necessary React hooks: `createContext`, `useContext`, `useState`
3. Create a `LanguageContext` using `createContext()`
4. Create a `LanguageProvider` component that:
 - Manages language state (default: 'en' for English)
 - Provides `language` and `setLanguage` to children via Context
5. Create a `useLanguage` custom hook that:
 - Uses `useContext` to access the `LanguageContext`
 - Throws an error if used outside `LanguageProvider`
 - Returns `{ language, setLanguage }`

Expected Code Structure:

```
import React, { createContext, useContext, useState } from 'react';

const LanguageContext = createContext();

export function LanguageProvider({ children }) {
  // Your code here
}

export function useLanguage() {
```

```
// Your code here  
}
```

Task 2: Integrate LanguageProvider in App.js

Location: `App.js`

Requirements:

1. Import `LanguageProvider` from `./contexts/LanguageContext`
2. Wrap the entire app with `LanguageProvider`
3. Ensure `LanguageProvider` wraps `EmailProvider` and `NavigationContainer`

Expected Structure:

```
<LanguageProvider>  
  <EmailProvider>  
    <NavigationContainer>  
      {/* Your app content */}  
    </NavigationContainer>  
  </EmailProvider>  
</LanguageProvider>
```

Task 3: Update SettingsScreen to Use Context

Location: `Screens/SettingsScreen.js`

Requirements:

1. Remove local `useState` for language
2. Import `useLanguage` hook from `./contexts/LanguageContext`
3. Use `useLanguage()` to get `language` and `setLanguage`
4. Pass these values to `LanguageSelector` component
5. Ensure the language selector works correctly

Before:

```
const [language, setLanguage] = useState('en');
```

After:

```
const { language, setLanguage } = useLanguage();
```

Task 4: Update LoginScreen to Use Context (Optional)

Location: `Screens/LoginScreen.js`

Requirements:

1. Remove local `useState` for language
 2. Import `useLanguage` hook from `../contexts/LanguageContext`
 3. Use `useLanguage()` to get `language` and `setLanguage`
 4. Pass these values to `LanguageSelector` component
 5. Verify that language selection in LoginScreen syncs with SettingsScreen
-

Testing Checklist

After completing the tasks, verify the following:

- ☐ `LanguageContext` is created and exports `LanguageProvider` and `useLanguage`
 - ☐ `App.js` wraps the app with `LanguageProvider`
 - ☐ `SettingsScreen` uses `useLanguage` hook instead of local state
 - ☐ Language selector in Settings page works correctly
 - ☐ Language selection persists when navigating between screens
 - ☐ Both `LoginScreen` and `SettingsScreen` show the same selected language
 - ☐ Changing language in Settings updates the selection in `LoginScreen` (if both have selectors)
-

Expected Behavior

1. **Initial State:** App starts with English ('en') as default language
 2. **Settings Page:** User can select English or Spanish using radio buttons
 3. **Persistence:** Selected language persists across all screens
 4. **Synchronization:** If language is changed in Settings, it updates everywhere
 5. **Context Access:** Any component can access the current language using `useLanguage()` hook
-

Hints

1. **Context Structure:** Follow the same pattern as `EmailContext.js`
2. **Provider Placement:** `LanguageProvider` should be at the top level (in `App.js`)
3. **Hook Usage:** Always use `useLanguage()` hook to access language state
4. **Error Handling:** The `useLanguage` hook should throw an error if used outside provider

Resources

- [React Context Documentation](#)
 - [React Context API Guide](#)
 - Review `contexts/EmailContext.js` for reference implementation
-

Good luck with your implementation! 🚀